

COMP101

Introduction to Programming

2025-26

Lecture-15

Iteration

General Theory

Iteration concept – ‘loop’

- Statements can be executed more than once
 - they ‘iterate’
 - We can repeat similar tasks several times
- Build an ‘iteration construct’ (a ‘loop’)
 - Has 2 parts
 - ‘loop header’ which may or may not include a test
 - ‘loop body’ with indented code
- Three different types of loop in general use
 - Python uses two of these

Iteration: Loop types (general)

- while loop (do ...while) #has a test for T/F
executes 0 or more times #'non-deterministic'
- for loop #has no test
executes always at least once # 'deterministic'
- repeat...until loop #not used in Python
executes always at least once

while loop

- ‘non-deterministic’
 - It is a ‘conditionally based loop’
- We don’t know how many times we need to execute the loop
- May not need to execute it at all
 - Depends on user input
- Evaluates a test in the loop header
 - evaluates True or False

while tests

- While test=True
 - Execute the loop body
 - Execute all indented code under the loop header
- While loop test=False
 - Ignore the loop body
 - execute the first un-indented statement that follows below the loop body

while: Structure

```
#1 import random
#2 play = input("Want to play a dice game? y/n ")
#3 while play == 'y':
#4     throw = random.randint(1,6)
#5     print ("You scored",throw)
#6     play = input ("Want to play again? y/n")
#7 print ("End of program")
```

Line numbering for guidance only.
Don't do this in your code – the whole line will become a comment

line#1,2,3 and 7

Sequence statements – must execute – not indented

Line#3

Must execute as it is a sequence statement.
It is also an iteration statement (a 'loop header') – where the loop starts because it begins with reserved word 'while' – Python expects a while-loop

line#4,5, 6

'loop body' – indented code that only executes if user input = 'y' at line#2
The loop starts and executes line#4,5,6. This continues until user input = 'n'
When user input = 'n' at line#6 we exit the loop and execute line#7
If user input = 'n' at line#2 we ignore line~3,4,5,6 and restart at line#7

line#8

end of while loop – it is not indented

While: test = True

```
#1 import random
#2 play = input("Want to play a dice game? y/n ")
#3 while play == 'y':
#4     throw = random.randint(1,6)
#5     print ("You scored",throw)
#6     play = input ("Want to play again? y/n")
#7 print ("End of program. Thank you")
```

Line#3 'sentinel' ('sentry') is y (from line#2) – start the loop – test is True

Line#4,5,6 The test is True - 'loop body' (line#4,5,6) executes in sequence:
The last line in the body MUST test the condition again
This line either makes the loop go round again or exits the loop

Line#7 Is not indented so indicates loop has ended or been ignored
Executes when user input = 'n' at line#6 or user input = 'n' at line#2

while: test = False

```
#1 import random
#2 play = input("Want to play a dice game? y/n ") n
#3 while play == 'y':
#4     throw = random.randint(1,6)
#5     print ("You scored",throw)
#6     play = input ("Want to play again? y/n")
#7 print ("End of program. Thank you")
```

Line#3	'sentinel' ('sentry') is n (from line#2) – test is False - do not do the loop
Line#4,5,6	ignore loop body as if line#4,5,6 don't exist ignore the entire loop, not just parts of it - loop has executed zero times
Line#7	Executes immediately after line#3 because user input = 'n' at line#2

while loop: Problem statement

“Students can choose to submit work for ‘formative assessment’ (unofficial grade).

Students who submit work are given a formative grade else no grade needs to be given”.

- We need a while-loop
 - ‘non-deterministic’ as number of submissions is unknown
 - If we know the number of submissions, use a for loop
- while can iterate zero or more times
 - If no one submits anything, the loop will not execute
- When loop does iterate, when do we stop it?
 - A for loop will end automatically when the last data is input

while loop construction

- Loop header
 - Has a test in it ('sentinel', 'sentry')
 - The sentinel allows loop to execute – or be ignored
 - test=T means start the loop
 - test=F means stop the loop – never executes
- Loop body
 - We need to know when to stop the loop
 - It is 'non-deterministic'
 - Use the 'sentinel' again to control the loop
- Execution continues at first un-indented line after the loop body

while loop construction - algorithm

Students can choose to submit work for 'formative assessment';
We don't know how many students (if any) will submit work;
Submissions will be given a formative grade else no grade given

pseudocode:

1. initialise
2. read in loop sentinel
3. loop
 - 3.1 read in grade
 - 3.2 read in loop sentinel
4. loopend
5. output grade list

#algorithm draft-1 (optional)

1. set loop sentinel
2. read in loop sentinel
3. loop while sentinel = T
 - 3.1 input(grade)
 - 3.2 read in loop sentinel
5. print grade list

Usual to code direct from pseudocode – language independent.
Sometimes for tricky bits intermediate algorithm drafts may help

while loop construction – algorithm draft-2

grade = ''	#input/initialize
grade = input (any grades? y/n)	
while grade = 'y'	#process
print (enter a student grade)	
grade = input (another grade? y/n)	#same test as above
print (grade list)	#output

Loop header: Has a test in it:

If user inputs 'Y' (test=T), execute the loop body

If user inputs 'N' (test=F) do not execute the loop body

Loop body:

Last line has same test in it (Y or N) that started the loop

Ask for same input of Y or N (can use different words in the prompt).

Y continues the loop and we go back to loop header, N exits the loop

while loop: draft-2 operation

```
#1 grade = ''                                # sentinel is space
#2 grade = input(any grades? y/n)            #sentinel is 'y' or 'n'

#3 while grade = 'y'                          #sentinel='y' do line#4; sentinel='n' do line#6
#4     input ('enter student score')
#5     grade = input ('another grade? y/n')  #sentinel='y' stay in loop do line#3,4,5
                                              #sentinel='n' exit loop - do line#3,6

#6 print (grade list)
```

for loop: when to use it

- ‘deterministic loop’
 - We know how many times to iterate because we know when we want the loop to end
- Always executes (‘iterates’) at least once and for the number of times we tell it to loop
- The number of loops is determined by the user

for loop: when to use it

- Controlled by a 'loop counter'
 - Counter must be 'initialized' before the loop starts
 - Never change it in the loop
 - changed automatically by the program
- It counts through the loop automatically using a sequence (1,2,3,4,5) or a range (1,5) of numbers
- Uses 'for var in ... '

for loop

```
import random
```

```
throws = int(input("How many throws do you want: "))
```

```
for i in range (throws):
```

```
    score = random.randint(1,6)
```

```
    print ("You scored",score)
```

```
print ("End of program. Thank you")
```

for loop: problem statement

“Students on a module receive an exam grade in range 0 to 100. Students who do not sit the exam are given zero”.

- We know how many students are on a module - so we know how many times to loop – there are no exceptions – process every student and then stop
- Needs a for loop (‘deterministic’) - we know how many students there are

pseudocode:

assign a grade to every student

1. initialise
2. read in number of students
3. loop number of students
 - 3.1 read in grade
4. loopend
5. output grade list

algorithm draft-1

may not need this but useful if tricky bits in the pseudocode

#no initialise

1. input (students)
2. loop for i = 1 to students
 - 2.1 input(grade)
3. print grade list

for loop – using the counter

```
students = integer
```

```
counter = 1
```

```
for counter in (students)
```

```
    students[counter] = grade
```

```
print (grade list)
```

```
#assume input = 10
```

```
#loop uses 'in' so we must  
compensate for start at [0]
```

```
#loop header – execute 10 times
```

```
#loop body – input a grade
```

```
#loop ended – no more indents
```

#operation

When loop starts, counter is set to 1 and we can prompt to input grade for student[1]

Execution returns to the loop header and the counter is automatically incremented by 1 (we don't do it ourselves)

Loop stops when student[10] grade is input

repeat loop – NOT IN PYTHON

- ‘non-deterministic’
 - Always executes at least once (‘iterates’)
 - But we don’t know how many times to execute the loop after that
- Test is at the end of the loop
 - Because test is at the end the loop must execute at least once in order to show the test to the user

repeat loop will loop at least once

Not in Python

```
repeat  #loop header
    print ('Main Menu')
    print ('A: Option-A')
    print ('B: Option-B')
    print ('X: Option-X')
    print ('Enter an option: ')
until option == 'X'
```

The menu will appear on screen even if the user does not want to see it.

Because the test is at the end of the loop, the repeat loop always executes at least once.

User must input 'X' to remove it

Python loop comparison

while loop

```
import random
play = input("Play dice game? y/n ")

while play == 'y':
    throw = random.randint(1,6)
    print ("You scored",throw)
    play = input ("Play again? y/n")

print ("End of program")
```

for loop

```
import random
throws = int(input("How many throws"))

for i in range (throws):
    score = random.randint(1,6)
    print ("You scored",score)

print ("End of program. Thank you")
```